ПРАВИТЕЛЬСТВО САНКТ-ПЕТЕРБУРГА
КОМИТЕТ ПО НАУКЕ И ВЫСШЕЙ ШКОЛЕ
СПб ГБПОУ «Петровский колледж»
Отделение информационных технологий

КУРСОВОЙ ПРОЕКТ

ПМ.02 Сопровождение и схемотехническое обслуживание интеллектуальных интегрированных систем

МДК.02.__ [заполнить]

Разработка и сопровождение виртуальной системы сбора и учёта данных на Arduino

Вариант №20 — «Газовая котельная»

ЧЕРНОВИК. Перед сдачей выполнить docs/BEFORE_SUBMISSION.md.

Выполнил: [ФИО], группа [группа]

Специальность: 09.02.08 «Интеллектуальные интегрированные системы»

Руководитель: [ФИО]

Санкт-Петербург, 2026

Задание и содержание

Вариант 20. Разработать виртуальную систему контроля газовой котельной: Arduino Uno, три датчика, реле и светодиод; подготовить SQLite, обработку логов в Node-RED и документацию.

Параметр	**Значение**
s1	MQ2, метан 0–100 %
s2	POT, давление газа 0–5 бар
s3	PIR, движение 0 или 1
Выходы	Реле (газовый клапан), белый LED 15 мА
Режим	T=5 с; порог s1>20 %

Дата выдачи: [заполнить]. Срок защиты: [заполнить]. Подписи — на исходном листе задания.

Содержание

Введение
1. Постановка задачи и структурная схема
2. Принципиальная схема и схема в Wokwi
3. Расчётная часть
4. Программа микроконтроллера
5. База данных
6. Обработка данных в Node-RED
7. Тестирование и проверка неисправности
8. Предложения по модернизации
Выводы, сокращения, литература
Приложения А–Ж

Введение

Контроль параметров газовой котельной требует согласованной работы датчиков, управляющего контроллера, средств индикации и регистрации событий. В учебном проекте эти задачи рассматриваются на полностью виртуальном стенде. Показания поступают от модели датчика MQ2, потенциометра и датчика движения PIR. Arduino Uno формирует текстовые сообщения, а программная часть сохраняет измерения и события в SQLite.

Цель проекта — разработать и описать цепочку сбора, передачи, проверки и хранения данных. Для её достижения требуется согласовать электрические подключения с прошивкой, определить формат сообщений, выполнить инженерные расчёты, подготовить базу данных и настроить загрузку лога через Node-RED. Отдельной задачей является обнаружение показаний, выходящих за допустимые пределы.

Результат работы представлен набором файлов: схемой Wokwi, скетчем, SQL-схемой и запросами, конфигурацией Node-RED, входными логами, диаграммами и инструкцией пользователя. Исходные материалы сохранены отдельно от предложенных исправлений и искусственных тестов. Это позволяет проверить происхождение данных и воспроизвести обработку.

Система предназначена исключительно для обучения. Процентная шкала MQ2 задаётся линейным преобразованием АЦП по условию задания; она не является калибровкой реального измерителя метана. Реле служит виртуальным исполнительным выходом. Сертифицированная автоматика газовой безопасности, реальные клапаны, аварийные цепи и эксплуатация газового оборудования в проект не входят.

1. Постановка задачи и структурная схема

Система должна опрашивать три канала с периодом 5 секунд. Канал s1 отражает условное содержание метана в диапазоне 0–100 %, канал s2 — давление газа в диапазоне 0–5 бар, канал s3 — наличие движения. При s1>20 % формируется тревога; при s1<=20 % система возвращается в нормальное состояние.

Измерительная часть состоит из MQ2 на A1, потенциометра на A2 и PIR на D2. Исполнительная часть включает реле на D5 и белый светодиод на D3. Скетч управляет их состоянием и передаёт сообщения через Serial Monitor. Перенос данных из браузерного симулятора в файл выполняется пользователем вручную: прямое соединение Wokwi с Node-RED не реализовано.

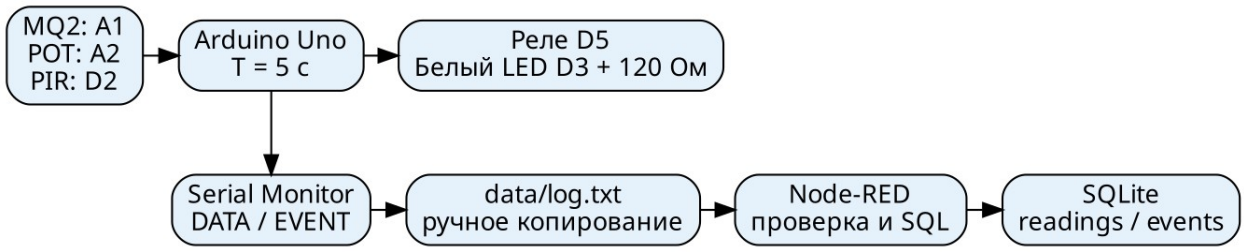


Рисунок 1 — Структура виртуальной системы

Node-RED читает файл целиком, разбирает формат, проверяет показания и формирует SQL-транзакцию. SQLite хранит три строки readings на каждое измерение и отдельные строки events для событий. Структурная схема показывает информационные связи и не заменяет принципиальную электрическую схему.

Критерии готовности: корректные DATA и EVENT, согласованные подключения, успешная загрузка базы, отсутствие дублей после повторного импорта, запись FAULT при недопустимом значении. Проверка виртуальных выходов и окончательная правка EasyEDA остаются обязательными перед защитой.

2. Принципиальная схема и схема в Wokwi

Проект Wokwi и скетч получены из присланного gas.zip. Ссылка: https://wokwi.com/projects/477934511917259777. Соединения diagram.json соответствуют указанным ниже выводам прошивки. В Wokwi установлен белый светодиод и резистор 120 Ом.

Элемент	**Назначение**	**Вывод**
MQ2	Учебная шкала метана 0–100 %	AO → A1
POT	Давление 0–5 бар	SIG → A2
PIR	Движение 0/1	OUT → D2
Реле	Имитация клапана	IN → D5
LED + R1	Индикация тревоги; 120 Ом	D3 → R → A; K → GND
Питание модулей	Общий источник питания и земля	5V и GND

Таблица 1 — Назначение элементов и выводы Arduino

В комплект включён новый экспорт EasyEDA — Schematic_kyrsovaya_2026-10-22.pdf. Номинал R1 на схеме и в новом BOM — 120 Ом; он соответствует расчёту и Wokwi. До сдачи необходимо сверить электрические сети в исходном EasyEDA: ползунок PR1 → A2, крайние выводы → 5V и GND; D3 → R1 → анод LED, катод → GND. Связи следует проверять выделением сети, а не только по пересечениям линий на PDF.

Файл bom.csv подготовлен из нового экспорта BOM_kyrsovaya_2026-10-22.csv: UTF-16 с табуляцией преобразован в UTF-8 CSV без изменения значений. Старые версии и прежний предложенный BOM сохранены только в docs/originals. Электрические связи и полярность LED проверяются по схеме, а не по BOM.

Контакты нагрузки реле в исходной модели не подключены. В проекте демонстрируется управление модулем; физический газовый клапан не моделируется. Схемы вынесены в приложения Б и В.

3. Расчётная часть

3.1. Резистор светодиода

Для белого индикатора по методичке Uf=3,2 В, ток I=15 мА=0,015 А. Напряжение питания — 5 В.

R=(5−Uf)/I=(5−3,2)/0,015=120 Ом.

120 Ом входит в заданный ряд номиналов, дополнительное округление не требуется. Расчётный ток равен 15 мА. Номинал 120 Ом теперь указан и в Wokwi, и в новом PDF EasyEDA, и в новом BOM. Дальнейшая проверка касается подключений и фактической работы, а не замены резистора.

3.2. Шаг АЦП

ΔU=5/1024=0,0048828125 В≈4,88 мВ.

Для s1 с диапазоном 0–100 %: Δs1=(100−0)/1023≈0,09775 процентного пункта на отсчёт.

Различие делителей обусловлено методичкой: 1024 уровня для шага по напряжению и диапазон кодов 0…1023 для линейного пересчёта датчика. Это оценка разрешения, а не точности реального MQ2.

3.3. Объём данных

Число циклов за сутки: 86400/5=17 280. Каждый цикл создаёт три записи readings.

N=(86400/5)×3=51 840 записей/сутки.

V=N×40=2 073 600 байт≈2,0736 МБ/сутки.

За 30 суток: 62 208 000 байт≈62,208 МБ. Здесь 1 МБ=1 000 000 байт. Размер строки 40 байт принят условно; индексы, служебные страницы SQLite и таблица events в оценку не включены.

4. Программа микроконтроллера

Скетч сохранён без изменений. В setup задаются Serial 9600 бод, входы и выключенные выходы. Реле в коде активно при LOW; подтвердите это по контактам модели Wokwi.

В loop читаются датчики и печатается DATA. EVENT выдаётся только при изменении alarm. Затем выполняется delay(5000). Первая DATA печатается сразу после setup; точный период включает время обработки.

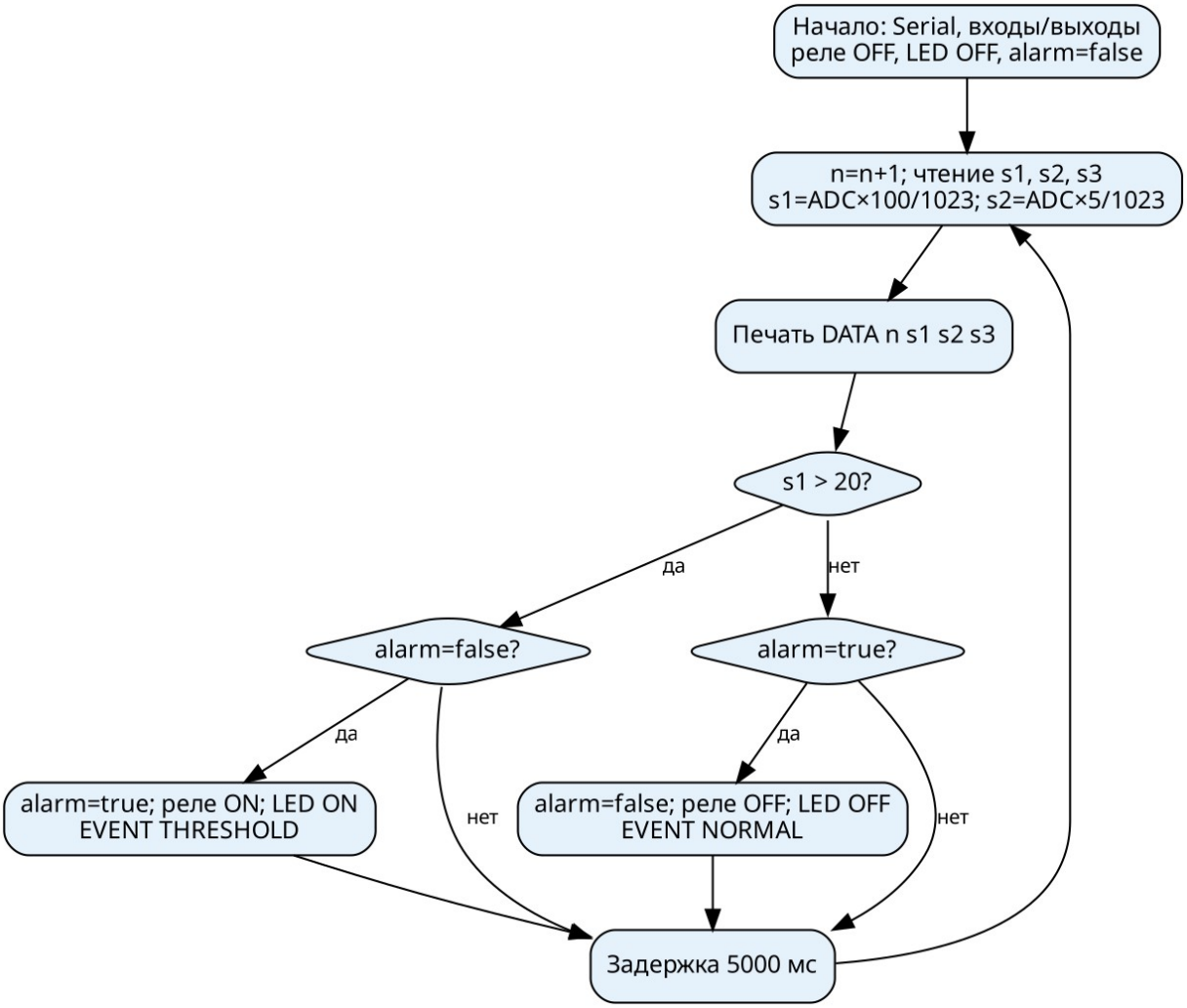


Рисунок 2 — Алгоритм исходного скетча

```
DATA n=2 s1=29.33 s2=2.50 s3=1
EVENT n=2 type=THRESHOLD
```

Формат: десятичная точка, максимум два знака после неё; n — целый номер измерения, s3 — 0 или 1. Полный скетч находится в firmware/sketch.ino и приложении Г.

5. База данных

Используется SQLite — файловая реляционная база без отдельного сервера. Подготовлена схема с таблицами readings и events. Выданный преподавателем schema.sql среди вложений отсутствовал, поэтому перед сдачей необходимо согласовать подготовленную структуру с преподавателем.

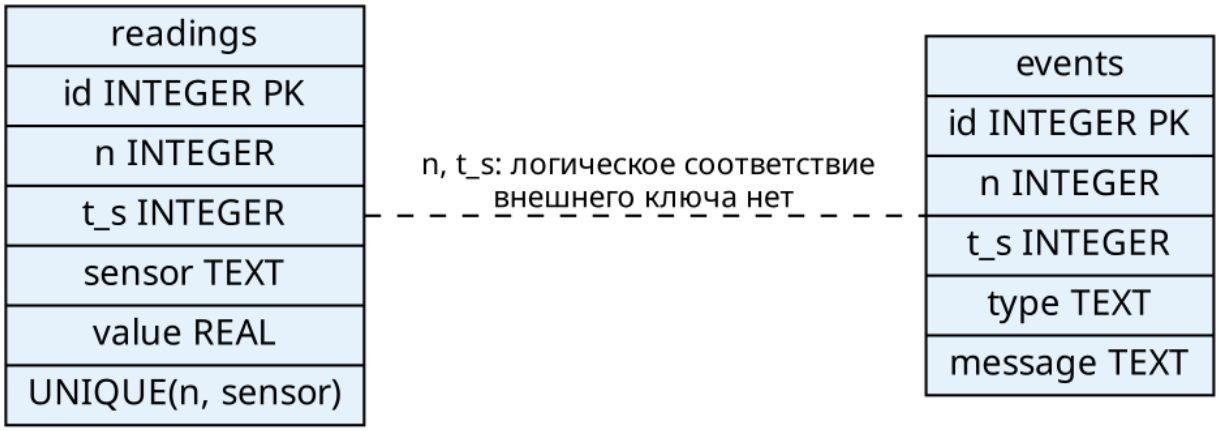


Рисунок 3 — Структура базы данных

readings содержит id, n, t_s, sensor и value. На каждое измерение создаются три строки, по одной для s1, s2 и s3. Ограничение UNIQUE(n,sensor) запрещает повтор одного канала в одном измерении. Недопустимые значения намеренно сохраняются: они нужны для диагностики FAULT.

events содержит id, n, t_s, type и message. Допустимые типы: THRESHOLD, NORMAL и FAULT. Таблицы логически сопоставляются по n и t_s. Внешнего ключа между ними нет: одному циклу соответствуют три показания и произвольное количество диагностических событий.

Временная отметка вычисляется как $t_s=n\times T$, в соответствии с шаблоном методички. Это условное время измерений, не календарный timestamp. В db/queries.sql сохранены запросы на десять последних строк показаний, MIN/AVG/MAX по каналам и все события по порядку.

В variant.db загружен присланный лог: 63 цикла, 189 строк readings и один THRESHOLD. Искусственные данные в итоговой базе не оставлены.

6. Обработка данных в Node-RED

Flow — nodered/flow.json. start-nodered.bat задаёт корень проекта для относительных путей. Редактор — 127.0.0.1:1880; используются только стандартные узлы.

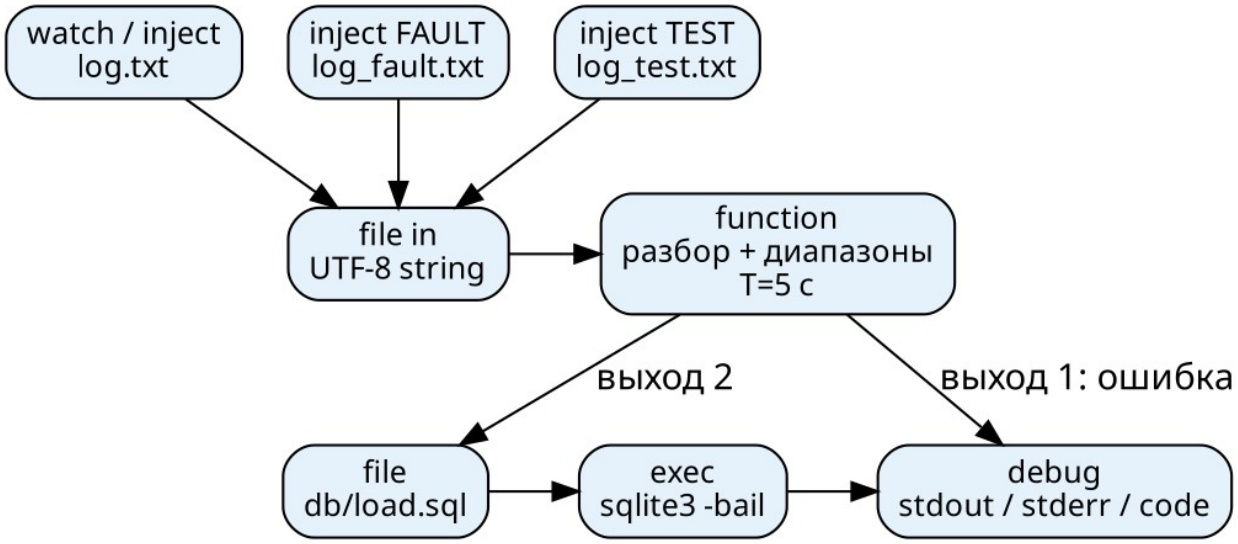


Рисунок 4 — Схема обработки; не скриншот редактора

watch реагирует на сохранение log.txt; file in читает строку UTF-8. inject запускает ручную загрузку или искусственный тест. Все ветви используют общий парсер.

function проверяет формат и уникальность DATA по n. Диапазоны: s1=0–100, s2=0–5, s3=0/1. Выход 1 — ошибка; выход 2 — SQL. Нарушение диапазона создаёт FAULT: «датчик sN вне диапазона».

file перезаписывает load.sql; exec вызывает sqlite3 -bail, читает schema.sql и load.sql. Транзакция заменяет данные целиком без дублей. stdout, stderr и код направлены в Debug; успешный код — 0.

В среде подготовки flow проверен на Linux с адаптером SQLite Python. Затем пользователь выполнил проверку на macOS: в SQLite появился FAULT, после загрузки основного лога — THRESHOLD. Материалы приведены в приложении Ж. Windows-запуск не проверен.

7. Тестирование и проверка неисправности

Ниже отделены фактические программные проверки от ещё не выполненных испытаний Wokwi. Искусственные DATA/EVENT проверяют импорт, но сами по себе не доказывают правильность работы Arduino.

Проверка	Ожидание	Фактически
Присланный log.txt	63×3 показания, 1 THRESHOLD	PASS: 189 readings, 1 событие
Повторный импорт	Нет дублей	PASS: состав тестовой базы не изменился
log_fault.txt: s1=101	FAULT для s1	PASS: подтверждено на Mac; скриншот
log_test.txt	Импорт THRESHOLD и NORMAL	PASS: 12 readings, 2 события
Неверный формат/NaN	Отказ без замены базы	PASS: парсер отклонил вход
Mac: возврат к log.txt	THRESHOLD вместо FAULT	PASS: вывод SELECT пользователя
Код скетча на хосте	Однократные переходы	PASS: C++-заглушки, не Wokwi
Порог и выходы в Wokwi	Реле и LED меняют состояние	Не выполнено; нужен реальный запуск
Windows + sqlite3.exe	Полная загрузка flow	Не выполнено; проверить у пользователя

Таблица 2 — Проверки и фактические результаты

Исходный лог содержит неизменное s1=88,66, давление 0 и отсутствие движения. В нём нет NORMAL, поэтому возврат к норме в Wokwi пока не подтверждён. Требуется уменьшить показание s1 до <=20 %, получить новый реальный лог и сохранить скриншоты.

Локальные проверки сохранены в runtime_checks.json и файлах тестов. Подтверждения с Mac — docs/evidence/mac; скриншоты и вывод команд находятся в приложении Ж. FAULT создан искусственно; возврат к THRESHOLD подтверждён текстом пользователя.

8. Предложения по модернизации

8.1. Устойчивое управление по порогу

При реальном шуме показаний значение может многократно пересекать 20 %, вызывая частое переключение выходов. Возможное развитие — гистерезис с разными порогами включения и выключения, а также временное подтверждение тревоги несколькими измерениями. В текущую реализацию это не добавлено: условие задания строго s1>20 %.

8.2. Накопление истории и контроль загрузки

Текущий flow каждый раз заменяет содержимое таблиц. Для накопления длительной истории можно добавить идентификатор сеанса и уникальный ключ «сеанс, n, sensor». После этого повторный импорт следует выполнять без дублей и без удаления старых сеансов. Полезны журнал импортов, отметка времени и сведения об источнике файла.

Для больших логов целесообразны ограничение размера файла, последовательная очередь импортов и потоковый разбор. Сейчас файл читается целиком, а кнопки загрузки не следует нажимать одновременно. Контроль диапазона не выявляет «застывший» датчик, если его значение остаётся внутри разрешённых границ; для этого нужен отдельный алгоритм.

8.3. Передача данных и диагностика

Ручное копирование Serial Monitor можно заменить поддерживаемым каналом передачи данных и подтверждением доставки. Также можно добавить проверку пропусков номеров, контроль периодичности, резервное копирование SQLite и просмотр истории. Конкретный способ подключения выбирается после уточнения среды выполнения.

Реальная газовая система требует профессионального проекта и сертифицированной автоматики. Учебная модель не пригодна для эксплуатации котельной.

Выводы

Подготовлена структура виртуального проекта варианта 20, сохранены исходные Wokwi-схема и прошивка, рассчитаны номинал резистора, шаг АЦП и условный объём данных. Созданы SQLite-схема, база с предоставленным логом, запросы и flow Node-RED. Проверена загрузка исходных и искусственных тестовых данных, запись FAULT и отсутствие дублей при повторном импорте.

Локальные испытания и материалы пользователя с macOS подтвердили программную обработку FAULT и возврат к исходному логу. Они не заменяют проверки Wokwi и Windows. До сдачи необходимо сверить сети EasyEDA, подтвердить выходы и реальный NORMAL, заполнить персональные данные и оформить Git. Записка остаётся черновиком.

Список сокращений

АЦП — аналого-цифровой преобразователь; БД — база данных; LED — светодиод; PIR — пассивный инфракрасный датчик; SQL — язык структурированных запросов; BOM — перечень элементов.

Список используемой литературы

1. Документация Wokwi. https://docs.wokwi.com
2. Справочник Arduino. https://www.arduino.cc/reference/
3. Документация Node-RED. https://nodered.org/docs/
4. Документация SQLite. https://www.sqlite.org/docs.html

Дополнительный источник требований: выданный документ «Титульник и задание ПМ 02». Исходные материалы: gas.zip, data.txt, PDF EasyEDA, BOM и скриншот пользователя.

Приложение А. Структурная схема

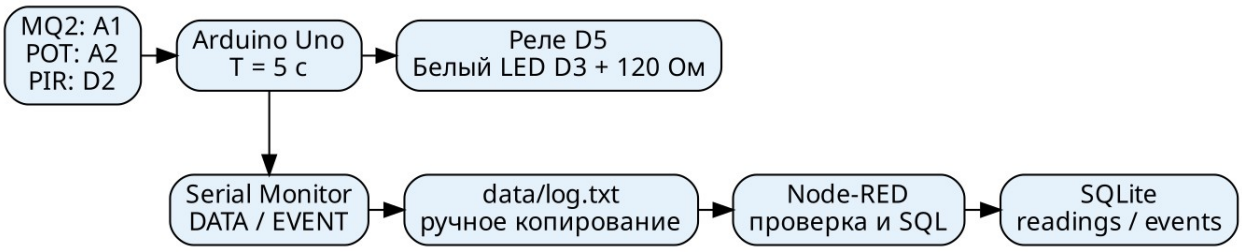


Рисунок А.1 — Передача данных и управляющие выходы

Оригинал диаграммы: docs/structure.dot; изображение: docs/structure.png. Перенос Serial → log.txt выполняется вручную. Параметры: T=5 с; тревога при s1>20 %.

Приложение Б. Принципиальная схема

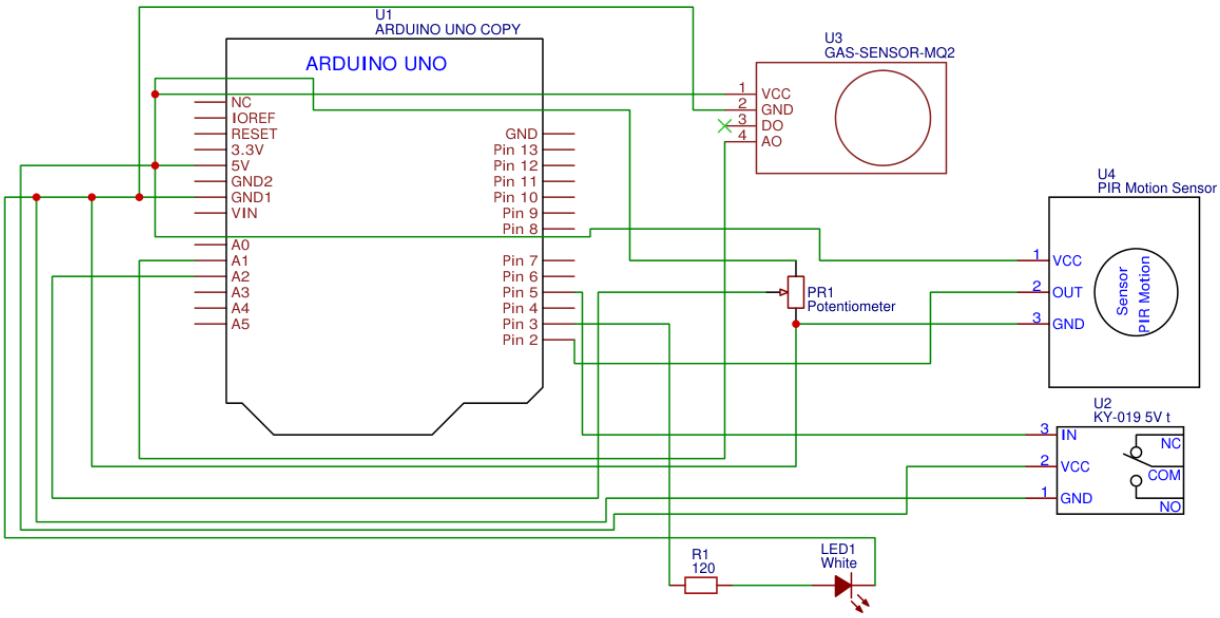


Рисунок Б.1 — Новый экспорт EasyEDA; R1 = 120 Ом

PDF заменён новым файлом пользователя без редактирования его содержимого. R1 = 120 Ом; новый перечень — bom.csv. Перед сдачей сверить сети: PR1 — ползунок на A2, крайние на 5V и GND; LED — анод через R1 к D3, катод к GND. Старый PDF находится только в docs/originals.

Приложение В. Схема в Wokwi

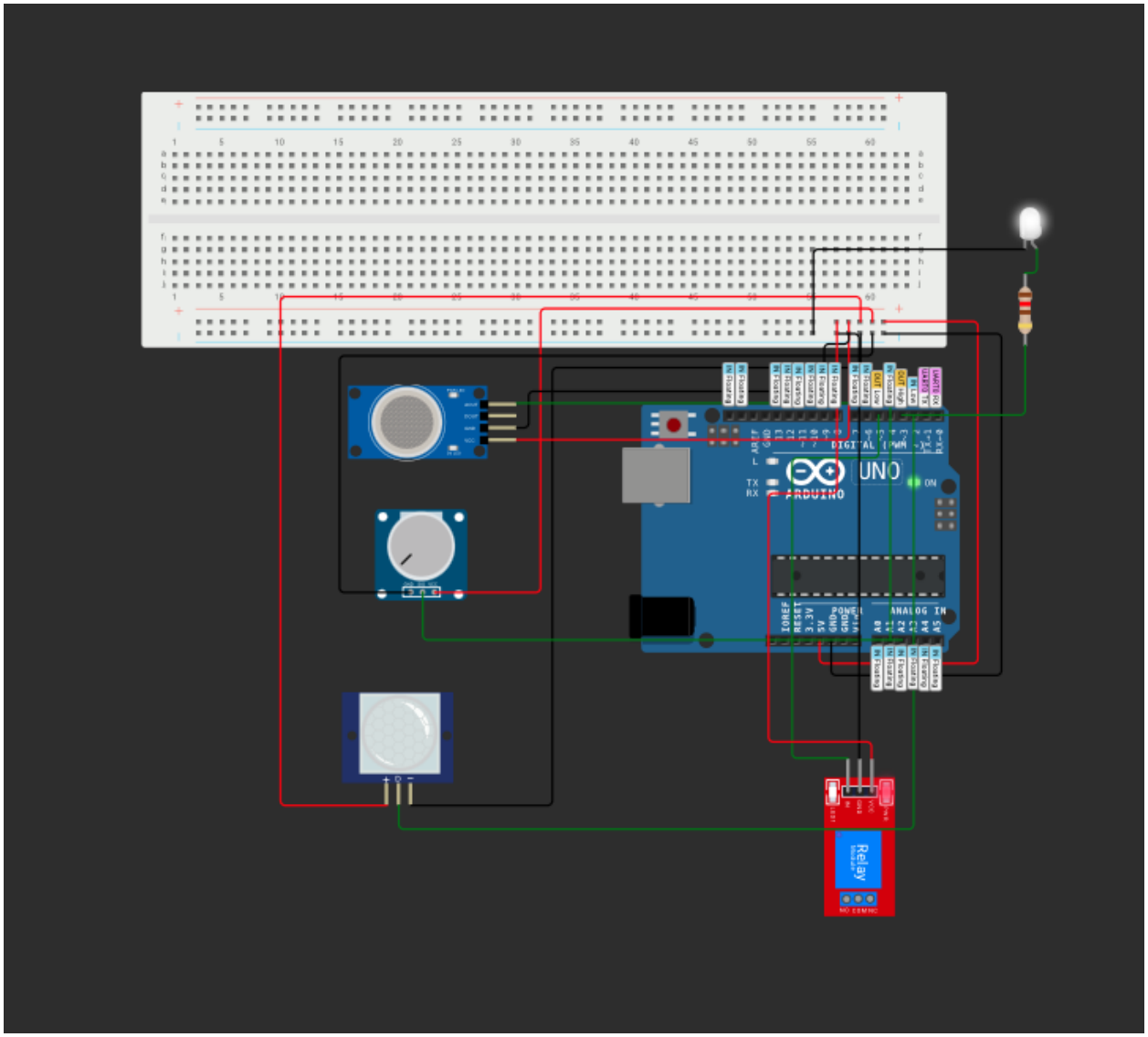


Рисунок В.1 — Предоставленный скриншот Wokwi

Схема diagram.json и скетч должны находиться в одном проекте Wokwi. Файлы в архиве разделены по структуре задания: sim/ и firmware/.

Приложение Г. Листинг скетча

```
const unsigned long T_MS = 5000;

const int PIN_S1 = A1;
const int PIN_S2 = A2;
const int PIN_S3 = 2;
const int PIN_RELAY = 5;
const int PIN_LED = 3;

const float THRESHOLD = 20.0;


const int RELAY_ON = LOW;
const int RELAY_OFF = HIGH;

unsigned long n = 0;
bool alarm = false;

void setup() {
  Serial.begin(9600);

  pinMode(PIN_S1, INPUT);
  pinMode(PIN_S2, INPUT);
  pinMode(PIN_S3, INPUT);


  digitalWrite(PIN_RELAY, RELAY_OFF);
  pinMode(PIN_RELAY, OUTPUT);

  digitalWrite(PIN_LED, LOW);
  pinMode(PIN_LED, OUTPUT);
}

void loop() {
  n++;


  float s1 = 100.0 * analogRead(PIN_S1) / 1023.0;


  float s2 = 5.0 * analogRead(PIN_S2) / 1023.0;


  int s3 = digitalRead(PIN_S3);
```

Приложение Г. Листинг скетча (продолжение)

```
  Serial.print("DATA n=");
  Serial.print(n);
  Serial.print(" s1=");
  Serial.print(s1, 2);
  Serial.print(" s2=");
  Serial.print(s2, 2);
  Serial.print(" s3=");
  Serial.println(s3);

  bool condition = s1 > THRESHOLD;

  if (condition && !alarm) {
    alarm = true;

    digitalWrite(PIN_RELAY, RELAY_ON);
    digitalWrite(PIN_LED, HIGH);

    Serial.print("EVENT n=");
    Serial.print(n);
    Serial.println(" type=THRESHOLD");
  } else if (!condition && alarm) {
    alarm = false;

    digitalWrite(PIN_RELAY, RELAY_OFF);
    digitalWrite(PIN_LED, LOW);

    Serial.print("EVENT n=");
    Serial.print(n);
    Serial.println(" type=NORMAL");
  }

  delay(T_MS);
}
```

Исходник сохранён без изменений. Проверка хостовой логики использует заглушки Arduino и не доказывает компиляцию под ATmega328P. Необходим реальный запуск в Wokwi.

Приложение Д. База данных

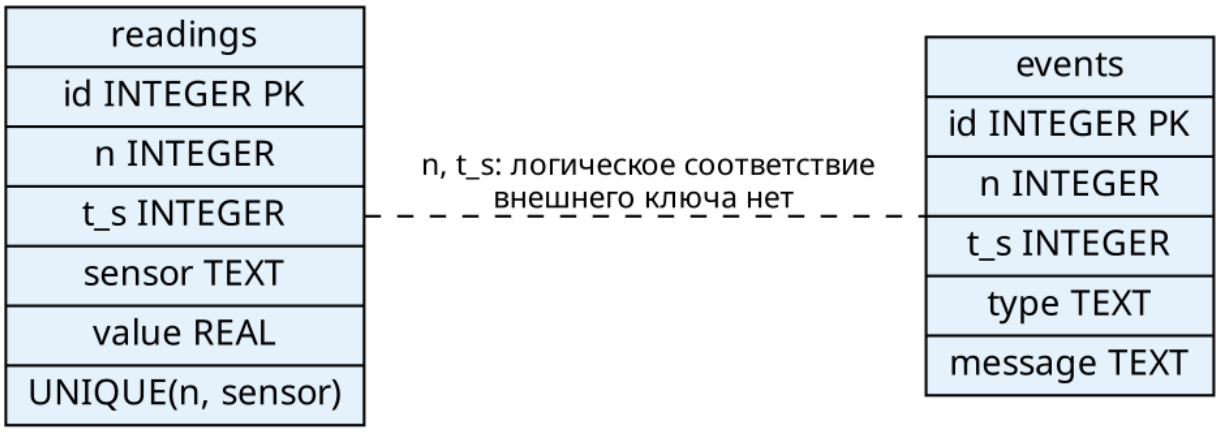


Рисунок Д.1 — ER-представление подготовленной схемы

```
-- Variant 20. Prepared schema; replace only after reconciling with teacher's schema.
PRAGMA foreign_keys = ON;
CREATE TABLE IF NOT EXISTS readings (
 id INTEGER PRIMARY KEY AUTOINCREMENT,
 n INTEGER NOT NULL CHECK(n>0),
 t_s INTEGER NOT NULL CHECK(t_s>=0),
 sensor TEXT NOT NULL CHECK(sensor IN ('s1','s2','s3')),
 value REAL NOT NULL,
 UNIQUE(n,sensor)
);
CREATE TABLE IF NOT EXISTS events (
 id INTEGER PRIMARY KEY AUTOINCREMENT,
 n INTEGER NOT NULL CHECK(n>0),
 t_s INTEGER NOT NULL CHECK(t_s>=0),
 type TEXT NOT NULL CHECK(type IN ('THRESHOLD','NORMAL','FAULT')),
 message TEXT NOT NULL DEFAULT ''
);
CREATE INDEX IF NOT EXISTS idx_readings_sensor_time ON readings(sensor,t_s);
CREATE INDEX IF NOT EXISTS idx_events_time ON events(t_s,id);
```

SQL-запросы — db/queries.sql. Заполненная база — db/variant.db. SQL последней загрузки — db/load.sql. Внешнего ключа между readings и events нет.

Приложение Е. Flow Node-RED

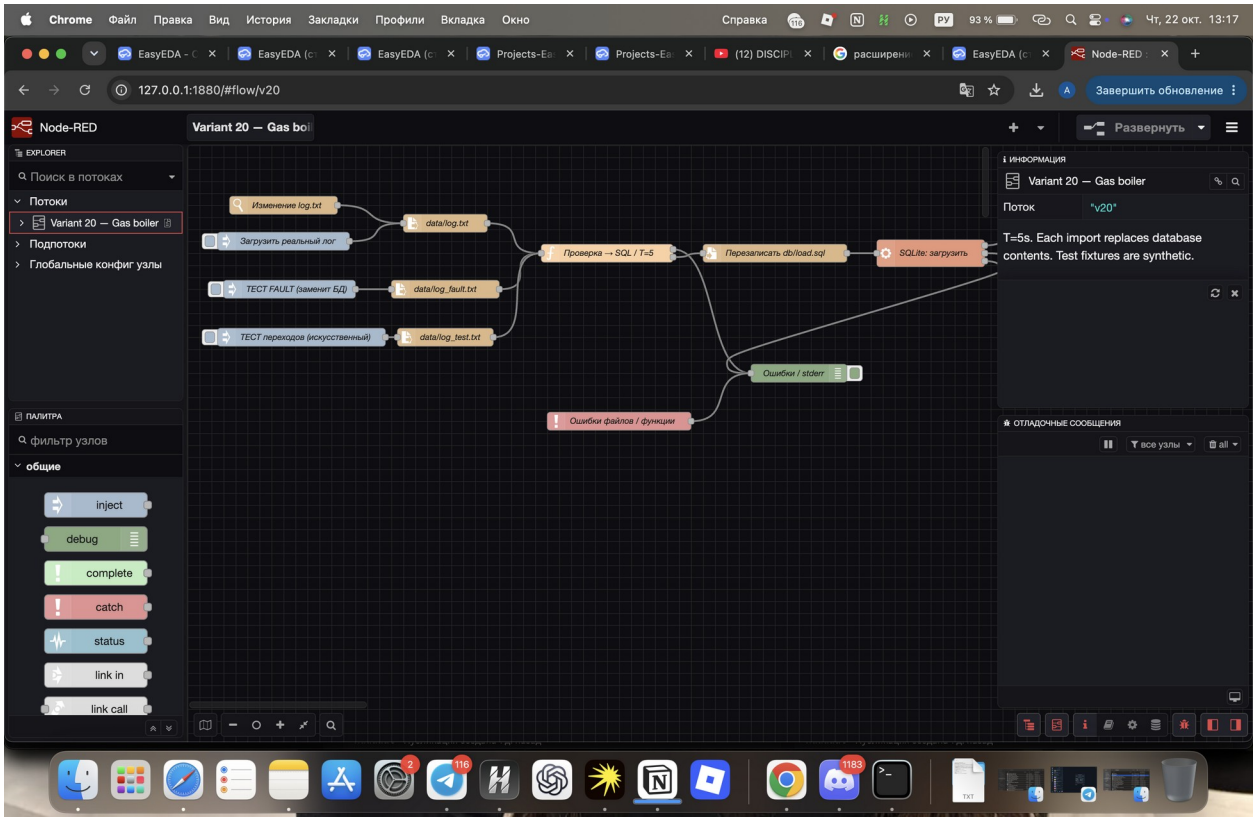


Рисунок Е.1 — Редактор Node-RED на Мас пользователя

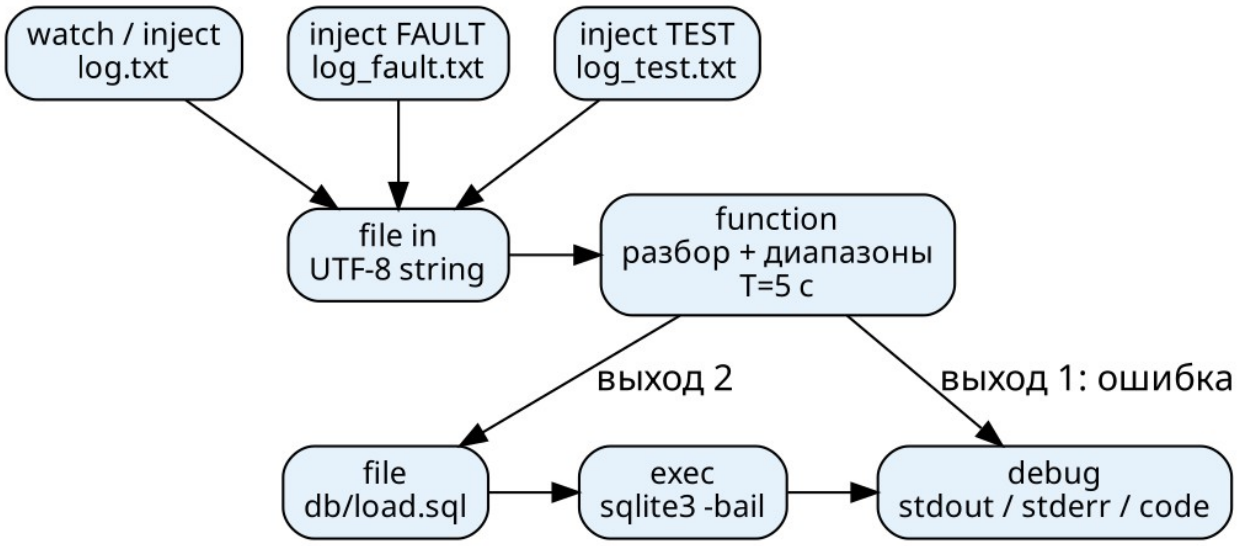


Рисунок Е.2 — Укрупнённая схема обработки

Конфигурация — nodered/flow.json; код функции — nodered/parser.js. Скриншот предоставлен пользователем: часть flow вне видимой области, Debug пуст. Результат FAULT и восстановление исходного лога подтверждаются материалами приложения Ж, а не этим снимком.

Приложение Ж. Проверка на macOS

Материалы предоставлены пользователем после запуска Node-RED на Mac. В искусственном тесте s1=101 получено событие FAULT; текст результата показан ниже.

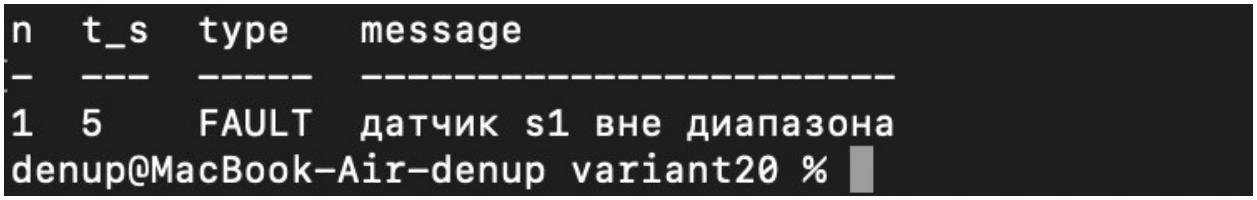


Рисунок Ж.1 — Результат FAULT; фрагмент скриншота пользователя

Исходный полный скриншот сохранён без изменений: docs/evidence/mac/terminal_fault_mac_original.png. Фрагмент обрезан только для читабельности. Первоначальная ошибка «cannot open db/queries.sql» исчезла после перехода в папку проекта.

После повторной загрузки основного log.txt пользователь прислал вывод следующего SELECT:

```
n  t_s  type       message
1  5    THRESHOLD
```

Этот последний результат получен из текста сообщения; на скриншоте он отсутствует. Он подтверждает возврат к основному логу после FAULT, но не переход NORMAL в Wokwi. Полный протокол — docs/evidence/mac/terminal_session_mac.txt.

Файл базы с Mac не передавался. В архиве оставлена ранее подготовленная variant.db с исходным логом. Проверка на Windows, реальные испытания выходов Wokwi и возврат NORMAL по-прежнему не подтверждены.